

Relatório Técnico – Etapa 2

1. Introdução

Nesta segunda etapa do projeto, o banco de dados foi expandido com o objetivo de automatizar regras de negócio, garantir a integridade das informações e oferecer suporte a consultas mais elaboradas. Para isso, foram utilizados recursos do PostgreSQL, como Stored Procedures, Triggers e Views, além da integração com uma ORM para facilitar a comunicação entre a aplicação e o banco de dados. Também foram implementados mecanismos de controle de transações e concorrência, tornando o sistema mais seguro e consistente em cenários de acesso simultâneo.

2. Stored Procedures

As Stored Procedures foram utilizadas para centralizar operações de negócio iniciadas pela aplicação, reunindo em um único local processos que envolvem múltiplas etapas e exigem execução segura dentro de transações.

A procedure `sp_registrar_atendimento_completo` realiza o cadastro de um atendimento juntamente com seus procedimentos em uma única transação. Os procedimentos são recebidos em formato JSONB e, caso alguma inconsistência seja encontrada, toda a operação é revertida, evitando registros incompletos.

A procedure `sp_calcular_tempo_medio_espera` calcula o tempo médio entre a chegada do paciente e o início do primeiro procedimento realizado em cada unidade hospitalar, utilizando um cursor (REFCURSOR) para retornar os resultados.

Por fim, a procedure `sp_reajustar_escala` permite alterar automaticamente as escalas de um residente entre datas e turnos, realizando verificações para impedir conflitos antes de efetuar a atualização.

3. Triggers

Enquanto as procedures executam operações solicitadas pela aplicação, as Triggers foram utilizadas para automatizar validações que devem ocorrer sempre que determinados dados forem modificados, garantindo que essas regras sejam cumpridas independentemente da origem da alteração.

A trigger `trg_check_sobreposicao_escala` impede que um residente seja escalado para duas unidades diferentes no mesmo dia e turno. A trigger `trg_audita_atendimento` registra automaticamente inserções, alterações e exclusões na tabela de atendimentos, armazenando o usuário responsável, a data da operação e os dados antigos e novos em formato JSONB. Já a trigger `trg_atualiza_media_procedimentos` recalcula automaticamente a média de tempo de

execução de um procedimento sempre que um novo registro é inserido na tabela de procedimentos realizados.

4. Views

As Views foram criadas para simplificar consultas recorrentes e disponibilizar informações consolidadas sem a necessidade de consultas complexas na aplicação.

A view `vw_pacientes_internados` lista os pacientes que permanecem internados, considerando apenas a internação mais recente. A `vw_residentes_sem_supervisor` identifica residentes escalados sob supervisão de preceptores que não possuem a titulação exigida. Já a `vw_estatisticas_atendimentos_mensal` reúne estatísticas por unidade e mês, apresentando a quantidade de atendimentos, a duração média e o procedimento mais frequente no período.

5. ORM, Transações e Controle de Concorrência

Para a camada de persistência foi utilizada a SQLAlchemy, escolhida por sua ampla utilização no ecossistema Python, boa documentação e integração com o PostgreSQL. A ORM permite mapear as tabelas do banco para classes da aplicação, reduzindo a quantidade de SQL escrito manualmente e facilitando o gerenciamento de entidades, relacionamentos, sessões e transações. Com isso, as operações da primeira etapa puderam ser reimplementadas de forma mais organizada e com maior facilidade de manutenção.

Também foi implementado um cenário de concorrência para simular dois usuários tentando escalar o mesmo residente simultaneamente. Para evitar inconsistências, foi utilizado bloqueio pessimista (lock pessimista), fazendo com que a segunda transação aguarde a conclusão da primeira. Após a liberação do bloqueio, a trigger de verificação impede a criação de uma escala conflitante, garantindo a integridade dos dados.

6. Considerações Finais

A segunda etapa permitiu incorporar recursos avançados de banco de dados, como procedures, triggers, views, ORM e controle de concorrência. Optou-se por utilizar Stored Procedures para concentrar operações mais complexas e executadas sob demanda pela aplicação, enquanto as Triggers ficaram responsáveis por validações automáticas e auditoria dos dados, garantindo que essas regras fossem aplicadas em qualquer alteração realizada no banco. Em conjunto com a ORM, essas decisões resultaram em uma solução mais organizada, segura e preparada para operar de forma consistente mesmo em cenários de acesso simultâneo e consultas analíticas mais complexas.